

Build Your Own WhisprFlow

A from-scratch recipe for building a native macOS push-to-talk dictation app. Hold a key, speak, release — your words get transcribed and pasted into whatever app you’re using.

You can either **build it yourself** following the milestones below, or **paste the prompt in 0 into an AI coding agent** (Claude Code, Cursor, etc.) and let it build the whole thing for you.

0. The one-shot agent prompt

If you just want the app, copy this entire block into Claude Code (or similar) inside an empty folder:

Build me a native macOS menu-bar app called **WhisprFlow**. It’s a push-to-talk dictation tool: when I hold the **Fn** key, it records my microphone; when I release, it sends the audio to a speech-to-text API, then pastes the transcript into the currently focused app via a synthesized **Cmd+V**.

Constraints:

- Pure Swift + AppKit. No external Swift package dependencies. Build with SwiftPM.
- Single executable target, wrapped into a .app bundle via a Makefile (ad-hoc codesign with `codesign --sign -`).
- Status-bar icon only (no dock icon — set `LSUIElement` in `Info.plist`).
- Record at 16 kHz mono int16 PCM, wrap into a WAV Data blob.
- Transcription provider auto-selects: **Deepgram** (`nova-3`, `smart_format=true`) if `DEEPGRAM_API_KEY` is set, otherwise **OpenAI** (`gpt-4o-transcribe`, fall back to `whisper-1`).
- Optional cleanup pass: if an OpenAI key is present and `enableCleanup` is true, post the transcript to `gpt-4o-mini` to strip fillers and apply spoken commands like “new paragraph” / “comma”. Skip cleanup when only Deepgram is configured.
- Config lives in `~/Library/Application Support/WhisprFlow/config.json` with fields: `apiKey`, `deepgramApiKey`, `customVocabulary`, `dictionaryReplacements` (array of {word, replacement}), `transcriptionModel`, `cleanupModel`, `enableCleanup`. Env vars `OPE-NAI_API_KEY` / `DEEPGRAM_API_KEY` are fallbacks when the JSON fields are empty.
- Floating “pill” overlay at the bottom-center of the screen: tiny idle pill (28×6), expands to 88×30 while recording with 11 animated bars driven by live mic level, shows 3 pulsing dots while transcribing, red dot on error. Use `CALayer` inside a borderless transparent `NSPanel`. Animate size + corner radius together in one `CATransaction`.
- Paste flow: save current pasteboard, write transcript, synthesize **Cmd+V** via `CGEvent`, restore pasteboard after ~0.4s.
- Three TCC permissions required: Microphone, Accessibility, Input Monitoring. Request mic explicitly; the others trigger on first use.

Read the rest of `BUILD.md` for milestone-by-milestone acceptance tests, gotchas, and the exact file layout. Build it milestone by milestone and verify each one before moving on.

That’s it. The rest of this doc is for humans who want to understand what they’re building, or for the agent to reference as it works.

1. What you're building

WhisprFlow is a menu-bar dictation app. There's no main window — just an icon in the menu bar and a small black pill floating at the bottom of your screen.

The interaction is:

- 1 Hold **Fn**. The pill expands and shows live audio bars.
- 2 Talk.
- 3 Release **Fn**. The pill shrinks to 3 dots while transcription happens.
- 4 Your transcript appears in whatever text field you had focused.

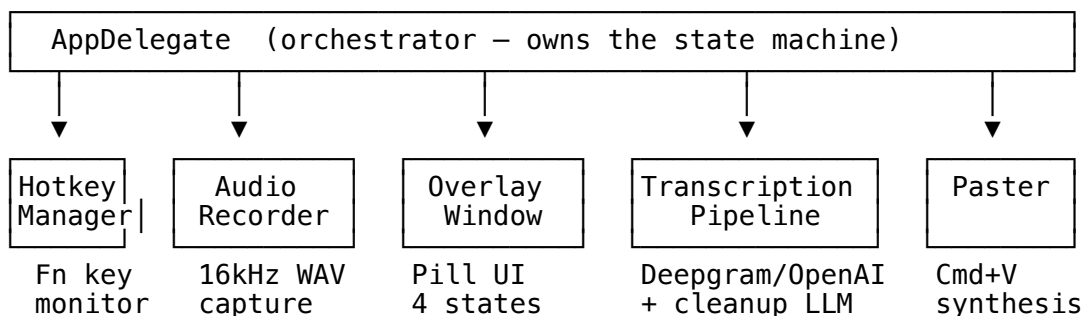
The whole loop takes ~1s of perceived latency with Deepgram. It feels like the OS just understands you.

2. Prerequisites

- macOS 13 (Ventura) or newer
- Xcode command-line tools: `xcode-select --install`
- An API key from **one** of:
 - Deepgram — recommended, faster and cheaper
 - OpenAI — needed if you want the LLM cleanup pass

No Xcode IDE needed. No CocoaPods, SPM dependencies, or Homebrew packages.

3. Architecture in one picture



4. Project layout

```
WhisprFlow/
├── Makefile                # builds .app bundle, ad-hoc signs
├── Package.swift           # SwiftPM manifest, single executable target
├── Resources/
│   └── Info.plist          # LSUIElement=true, mic usage description, bundle ID
├── Sources/WhisprFlow/
│   └── main.swift          # UIApplication boot
```

```

├─ AppDelegate.swift      # state machine
├─ Config.swift          # JSON config singleton
├─ HotkeyManager.swift    # Fn key global monitor
├─ AudioRecorder.swift    # AVAudioEngine → WAV Data
├─ OverlayWindow.swift    # the floating pill
├─ TranscriptionPipeline.swift # Deepgram / OpenAI / cleanup
├─ Paster.swift          # clipboard + synthetic Cmd+V

```

5. Build order (milestones)

Build in this order. After each milestone, you should be able to **do the thing in the “Acceptance” line** before moving on. If you can’t, fix it before continuing — every later milestone assumes the earlier ones work.

M1 — Hello menu bar

- Package.swift with one executable target.
- main.swift boots NSApplication, sets .accessory activation policy, installs AppDelegate.
- AppDelegate creates an NSStatusItem with a system symbol (e.g. mic.fill).
- Makefile: swift build -c release, copy binary into WhisprFlow.app/Contents/MacOS/, copy Info.plist, run codesign --sign - --force --deep WhisprFlow.app.

Acceptance: make && open WhisprFlow.app puts a mic icon in your menu bar. No dock icon appears.

M2 — The pill (idle only)

- OverlayWindow: borderless transparent NSPanel, level = .statusBar, never key.
- Inside it, a CALayer rounded rectangle, 28×6, black with a 1px white border, positioned bottom-center of the main screen.
- AppDelegate shows it on launch.

Acceptance: A tiny black pill is always visible at the bottom of your screen.

M3 — Push-to-talk hotkey

- HotkeyManager: NSEvent.addGlobalMonitorForEvents(matching: .flagsChanged).
- Filter: only fire when event.keyCode == 63 (the Fn key). Distinguish press vs release by whether the .function modifier is currently set.
- Callbacks: onPress, onRelease.

Acceptance: NSLog “down” / “up” when you press and release Fn. **You’ll need to grant Input Monitoring + Accessibility in System Settings** the first time. (See § 6.)

M4 — Audio capture

- AudioRecorder: AVAudioEngine, tap the input node, convert to 16 kHz mono int16, accumulate samples in a Data buffer.
- On stop(), prepend a 44-byte WAV header and return the Data.
- Expose onLevel: (Float) -> Void fired on main with sqrt-normalized peak amplitude per buffer.

Acceptance: Hold Fn for 2 seconds, release, write the returned Data to /tmp/test.wav, open in Quick-

Time → you hear yourself.

M5 — Pill animates

- Add an `OverlayState` enum: `.idle`, `.recording`, `.transcribing`, `.error(String)`.
- On state change, animate the pill's `bounds.size` and `cornerRadius` together inside one `CATransaction` (~120 ms ease-out).
- `.recording`: 88×30, draw 11 white bars whose heights come from a rolling 11-slot buffer of `onLevel` values.
- `.transcribing`: 88×30, three pulsing white dots.
- `.error`: 88×30, single red dot, auto-revert to `.idle` after 2 s.

Acceptance: Holding Fn expands the pill smoothly and the bars dance with your voice.

M6 — Transcription (Deepgram path first)

- Config: load/save JSON at `~/Library/Application Support/WhisprFlow/config.json`. Env-var fallback for empty fields.
- `TranscriptionPipeline.transcribe(wav: Data) async throws -> String`.
- If `deepgramApiKey` is set: POST `https://api.deepgram.com/v1/listen?model=nova-3&smart_format=true&punctuate=true&filler_words=false` with header `Authorization: Token <KEY>`, body is raw WAV. Parse `results.channels[0].alternatives[0].transcript`.

Acceptance: Hold Fn, say “hello world”, release, NSLog prints `hello world`.

M7 — Paste it

- `Paster.paste(_ text: String):`
 - 1 Save current `NSPasteboard.general.string(forType: .string)`.
 - 2 Write text.
 - 3 Build two `CGEvents` for Cmd+V (key down + key up, source nil, virtual key 9, flag `.maskCommand`), post them to `.cghidEventTap`.
 - 4 After 0.4 s, restore the saved string.

Acceptance: Click into a TextEdit window, hold Fn, say something, release → it appears.

M8 — OpenAI fallback + cleanup

- If no Deepgram key but `apiKey` is set: multipart POST `/v1/audio/transcriptions` with `model=gpt-4o-transcribe`. On failure, retry with `whisper-1`.
- If `enableCleanup` **and** OpenAI key set: POST `/v1/chat/completions` with `gpt-4o-mini`, system prompt that strips fillers, applies spoken commands (“new paragraph”, “comma”), and honors `dictionaryReplacements`. Cleanup failure is non-fatal — return the raw transcript.
- Skip cleanup automatically when only Deepgram is configured.

Acceptance: With cleanup on, “uh hello world period new paragraph thanks” becomes `Hello world.\n\nThanks.`

M9 — Menu bar menu + dictionary panel

- Click the status item → menu with two items: **Dictionary...**, **Quit**.

- Dictionary opens an `NSWindow` with a table view bound to `Config.dictionaryReplacements` (add/remove rows). Save on close.

Acceptance: Add `{ "word": "claude", "replacement": "Claude" }`, dictate “i love claude”, paste shows I love Claude.

6. Gotchas (read these — they will bite)

- **TCC permissions reset on every rebuild.** `make` deletes the `.app` and ad-hoc re-signs it, which changes the bundle’s code identity. macOS treats every rebuild as a *different app* and silently revokes Microphone / Accessibility / Input Monitoring. After every `make`, re-grant all three in **System Settings** → **Privacy & Security**, then relaunch. (Fix: use a stable codesign identity instead of `--sign -`. Out of scope for this build.)
- **Always rebuild + install together.** Make your build command `make && ditto WhisprFlow.app /Applications/WhisprFlow.app` so the copy you launch isn’t stale.
- **Fn key is keyCode == 63.** Don’t be tempted to listen for `.function` in `event.modifierFlags` — arrow keys / Home / End / Page Up/Down all assert the function modifier but do **not** generate a `flagsChanged` event for `keyCode 63`. Filtering on `keyCode 63` gives you the physical Fn key only.
- **Env vars don’t reach the app when you open it.** `open WhisprFlow.app` is launched by `launchd`, which doesn’t read your shell `.env`. If you rely on `OPENAI_API_KEY` / `DEEPGRAM_API_KEY`, either set them in the JSON config or launch the binary directly: `./WhisprFlow.app/Contents/MacOS/WhisprFlow`.
- **Mic permission must be requested explicitly.** Call `AVCaptureDevice.requestAccess(for: .audio)` in `applicationDidFinishLaunching`. Accessibility + Input Monitoring trigger on first use only — there’s no API to request them.
- **Pasteboard restore is text-only.** If the user had an image or file on the clipboard, it’s lost. Document it; don’t try to be clever.

7. Stretch ideas (after the clone works)

- Replace ad-hoc codesign with a self-signed identity so TCC grants survive rebuilds.
- Stream Deepgram over WebSocket for partial transcripts as you speak.
- Add a “preview” mode where the transcript appears in the pill before pasting, with a key to cancel.
- Swap Fn for a configurable hotkey (any modifier or even a chord).
- Local Whisper.cpp provider for fully offline transcription.

8. If you’re handing this to an AI agent

Tell it: “Read `BUILD.md` end-to-end first. Build milestone by milestone. After each milestone, run the acceptance test and tell me the result before continuing. If a milestone fails, stop and ask me — don’t try to skip ahead.”

That single instruction is the difference between “agent ships a working app” and “agent generates 800 lines of plausible Swift that doesn’t compile.”